

Titre du travail de maturité ici sur deux lignes

TRAVAIL DE MATURITÉ

Auteur

Prénom Nom

Mentor

Prénom Nom

Gymnase de Bienne et du Jura bernois

Octobre 2026

Résumé

Remerciements

Merci à ...

Table des matières

Remerciements	2
1 Introduction	5
2 Développement	6
2.1 Internet et son fonctionnement	6
2.2 TCP	7
2.3 UPD	9
2.4 Attaques DOS	9
2.4.1 Attaque SYN Flood	9
2.4.2 UPD Flooding	9
2.4.3 Packet Fragment	10
2.5 Apache2	10
2.5.1 Effets des paramètres Apache2	10
3 Conclusion	11
A Annexe A : Données complémentaires	12

Table des figures

1 Introduction

À l'ère numérique, nous retrouvons des appareils électroniques partout autour de nous. Dans nos maisons, dans la rue, ou même encore dans les transports en commun. Nous utilisons tous ces appareils au quotidien : le téléphone, un ordinateur, une télévision, et encore. Sur certains de ces appareils, il existe une fonction que nous utilisons particulièrement : la recherche Internet. Nous recherchons des informations, voulons nous distraire ou encore communiquer.

Pour mon travail de maturité, je me suis intéressé aux sites web en particulier. Il existe un nombre incalculable de sites web auxquels nous pouvons accéder, chacun contenant une grande variété de contenus. Ces sites web ont tous un point commun : ils doivent être accessibles aux utilisateurs : c'est-à-dire à vous. Une question survient : comment un site peut-il être visité par des clients ?

Pour qu'un site soit disponible, il est nécessaire de l'héberger sur un serveur (web). J'ai donc choisi de m'intéresser particulièrement à ce point. Un serveur doit être capable, de recevoir la demande que le client envoie, comme un clic sur un lien d'une page web, demandant l'accès au site. Il doit non seulement être recevoir la demande, mais aussi renvoyer une réponse, dans notre cas, suite au clic, en fournissant les données nécessaires à l'ouverture du site. Comme vous devez le savoir, vous n'êtes souvent pas la seule personne voulant accéder à un site en particulier, il faut donc non seulement pouvoir recevoir et renvoyer une réponse mais en gérer plusieurs.

Quand un serveur reçoit et renvoie une réponse, il consomme des ressources (CPU, mémoire). Ces ressources ne sont pas illimitées. Si un grand nombre de demandes est envoyé, les ressources du serveur peuvent s'épuiser ; par conséquent, celui-ci ne sera plus capable de traiter toutes les demandes des clients. Le site sera donc inaccessible pour certains.

Nous arrivons ainsi à l'un des points principaux de mon travail de maturité : qu'est-ce qui peut volontairement provoquer l'arrivée d'un grand nombre de demandes ? Il existe des attaques permettant d'envoyer un grand nombre de demandes, appelées attaques DoS. Elles inondent le serveur pour saturer ses ressources. Ainsi le point que j'ai étudié, au long de mon travail de maturité :

comment provoque-t-on une attaque de type surcharge DoS et comment peut-on s'en protéger ?

Pour répondre à cette problématique, j'ai choisi d'adopter la méthode suivante : Premièrement, j'ai mis en place deux serveurs web sur une même machine. Sur ces serveurs web, j'ai créé une page web simple disponible localement. Deuxièmement, j'ai recherché un outil pour simuler une attaque de type DoS. Ensuite, j'ai analysé les performances de chacune des pages web pendant une simulation de DoS. J'ai progressivement augmenté la charge pour voir l'évolution des statistiques. Après avoir fait cela, j'ai effectué des recherches et changé les configurations des serveurs web pour voir leur efficacité et montrer quels en sont les impacts. Par ailleurs, j'ai comparé les caractéristiques des différents serveurs web pour déterminer lequel est le plus efficace. Pour finir, j'ai analysé les résultats obtenus lors de ces expériences.

2 Développement

2.1 Internet et son fonctionnement

Internet est un réseau connectant un grand nombre d'ordinateurs les uns avec les autres. Cette connexion se fait par des fils, comme la fibre optique, le cuivre (DSL) ou par la connexion aux satellites. Deux ordinateurs connectés à ce fil peuvent communiquer. Les serveurs sont des ordinateurs fournissant un service accessible par Internet. Les pages web sont des fichiers contenus dans le disque dur de ces serveurs. Chaque serveur possède une adresse de protocole différente ou, communément appelée, IP. Les adresses de protocole fonctionnent comme une adresse postale, les IP aident les serveurs à se trouver entre eux. Exemple d'IP : 72.14.201.100. Ces IP peuvent être assignées à des noms, comme google.com, outlook.com, etc.

L'ordinateur que vous possédez à la maison n'est pas un serveur, car il n'est pas connecté directement à Internet. Les ordinateurs que nous utilisons tous les jours sont appelés clients, car leur connexion à Internet est indirecte. Ils passent d'abord par un fournisseur d'accès Internet (ISP), qui lui va être connecté à Internet

Exemple :

Nous voulons visiter "outlook.com", qui est un service de messagerie hébergé sur des serveurs de Microsoft. Celui-ci est connecté à un ISP. Prétendons que nous utilisons la fibre optique. Notre ordinateur envoie une requête à notre ISP qui passe par Internet et ensuite arrive jusqu'aux serveurs de Microsoft. Si nous voulons envoyer un mail, et que nous utilisons gmail.com mais que le destinataire possède outlook.com. Notre ordinateur va envoyer la requête via notre ISP qui lui va contacter les serveurs de Gmail. Gmail va se connecter aux serveurs de Microsoft qui vont stocker les informations et les transmettre

au destinataire. Nous avons parlé d'envoyer des informations d'un point à l'autre, mais pas comment cette information est transmise. Quand un e-mail, une image ou une page web traverse Internet, les ordinateurs découpent l'information en plus petites pièces, elles sont appelées "paquets". Quand l'information arrive au destinataire, les paquets sont réassemblés dans le bon ordre pour reformer l'e-mail, l'image ou encore une page web.

Mais comment nos paquets ne se retrouvent-ils pas chez un autre client ? Comme nous l'avons précédemment dit, tout objet connecté directement ou indirectement à Internet possède une adresse IP. Entre chaque intersection d'Internet, un équipement appelé "routeur" est assigné. Ces routeurs interceptent les paquets et les dirigent ensuite vers le bon destinataire en fonction de l'adresse IP client et de l'adresse IP serveur. La requête passe par plusieurs routeurs avant d'arriver au destinataire. Pour répondre à la requête, le serveur destinataire envoie l'information de réponse avec les IP correspondantes à l'échange

2.2 TCP

Protocol TCP (Transmission Control Protocol) : Le protocole TCP est une norme de communication permettant à des programmes ou des dispositifs informatiques d'échanger des messages sur un réseau. C'est ce protocole qui permet d'assurer la transmission des données en toute sécurité. Le protocole TCP organise les données afin qu'elles puissent être transmises entre un client et un serveur. Avant de transmettre des données, TCP établit un lien entre la source et la destination, ce qui permet de vérifier si les données sont correctement reçues et de retransmettre celles qui seraient perdues. La combinaison des protocoles TCP et IP permet l'échange de données. C'est pour cela que nous parlons souvent de TCP/IP. C'est le processus complet qui permet aux utilisateurs de déplacer des données entre plusieurs appareils. Quand les données sont traitées par TCP/IP, elles doivent passer par 4 couches

Cette communication, ce fait en différentes couches, celle-ci permettent que les données soient correctement transmises et en toute sécurité. Une particularité de cette communication est que chacune des couches s'appuie sur les services offerts par la couche du dessous. Une autre particularité est l'interchangeabilité, cela signifie que nous pouvons remplacer une couche sans avoir à modifier les autres couches, et cela malgré le fait que les couches s'appuient sur les services de l'autre. Finalement, la complexité est moindre car chaque couche se concentre sur une tâche en particulier.

Pour mieux comprendre, prenons un exemple analogique :

Une lettre envoyée par la Poste. Premièrement, il faut écrire la lettre, avec la bonne formulation et on s'assure que la correspondance est bien établie.

Deuxièmement, le transport : la lettre est glissée dans l'enveloppe, une étiquette est ajoutée à l'enveloppe, celle-ci précise le service du destinataire.

Ensuite, le réseau : Il faut inscrire l'adresse exacte du destinataire, comme la rue, le code postal ou encore le numéro. Dans la même étape, la lettre est envoyée vers la ville de destination.

Ensuite vient la liaison des données : Le facteur doit choisir un chemin local pour accéder à la boîte aux lettres. Quand celui-ci est arrivé, il dépose la lettre portant le nom de la personne à la poste.

Pour finir, la lettre doit être transportée physiquement (vélo, voiture, camion).

Cette exemple analogique montre exactement comment marche la pile TCP/IP, Nous allons donc en parler avec des termes un peu plus scientifique.

La première couche, qui se nomme Application Layer fait référence à la première étape décrite précédemment, elle décrit qu'elle information peut être envoyées, celle-ci peuvent être décrites comme des pages HTML ou bien des requêtes HTTP.

Pour ce qui est de la deuxième couche, le transport Layer, cette couche permet d'assurer la communication de bout en bout entre les applications, de découper les données en segments et selon le protocole utilisé, garantir ou non l'ordre et la fiabilité.

Pour ce qui est du réseau que nous avons mentionnés dans l'exemple analogique, on l'appelle : Internet Layer. Dans cette couche, les paquets sont transportés à travers le réseau mondial (internet).

L'avant dernière couche se nomme "Link Layer", c'est le moment où la transmission se fait localement, par exemple par Wi-Fi, depuis un réseau local ou bien par Ethernet.

Et pour finir, le Physical Layer qui fait référence au transport dans l'exemple, est comment les bits sont physiquement transmis sur le support. On peut trouver comme exemple les ondes ou bien encore les câbles.

2.3 UDP

User Datagram Protocol ou plus communément appelé UDP est une alternative au protocole TCP. Néanmoins, le protocole UDP s'utilise sans connexion, il va envoyer les données d'un client vers une destination serveur, sans pour autant chercher à savoir si le destinataire a bien reçu les données. Formuler autrement, il n'y a pas de vérification d'erreurs. Le seul objectif du protocole UDP est d'envoyer les données. Puisqu'il n'y a pas de vérification, ce protocole est plus rapide que TCP. Quelques exemples de protocole qui utilise UDP :

- Protocole DNS
- Protocole TFTP (Transfert de fichiers simplifié)
- Protocole NTP (mise à jour de la date et l'heure via un réseau)

2.4 Attaques DOS

2.4.1 Attaque SYN Flood

C'est une attaque créant un déni de service en émettant un très grand nombre de demandes TCP incomplètes. Comme nous l'avons précédemment dit, le client et le serveur échangent une séquence de messages avec le TCP/IP. Le client envoie une demande SYN, le serveur répond par un message SYN-ACK et le client envoie un message ACK pour confirmer la réception du message SYN-ACK. Ce processus sert à initialiser le protocole TCP/IP. La faiblesse se trouve là où le serveur a envoyé une réponse SYN-ACK au système client. Si le client ne renvoie pas de message de confirmation ACK, le serveur reste en attente d'une réponse. Il stocke dans sa mémoire la demande SYN précédemment faite par le client. Aucun serveur ne possède des ressources illimitées. Si un grand nombre de demandes SYN sans réponse sont faites, le serveur finira par être saturé, ce qui provoquera un arrêt des fonctionnalités du serveur (page web, application, etc.). Bien sûr, il y a souvent un délai d'attente associé à une connexion entrante, les semi-connexions vont se fermer et le serveur sera en parfait état. Mais il suffit à l'attaquant d'envoyer plus de demandes que le serveur n'en éjecte. Le serveur finira surchargé malgré le fait que des connexions soient éjectées. source : Dos

2.4.2 UDP Flooding

Cette attaque exploite le fait qu'il n'y ait pas de connexion dans le protocole UDP. L'attaquant crée une grande quantité de paquets UDP soit à destination d'une machine, soit entre deux machines. Cela provoque une saturation des ressources des deux machines ainsi que celle du réseau. Cette attaque est plus conséquente car le protocole UDP est prioritaire sur la bande passante par rapport au protocole TCP, donc le protocole UDP finit par utiliser toutes les ressources du réseau, ne laissant pratiquement rien aux autres protocoles.

2.4.3 Packet Fragment

Le Packet Fragment utilise les faiblesses dans l'implémentation de certaines piles TCP/IP au niveau de la défragmentation IP. Exemple : Le Teardrop est une attaque connue utilisant le principe de Packet Fragment. Le Teardrop consiste à envoyer un fragment contenant des octets déjà envoyés par un autre fragment. La machine ne sait pas quel fragment utiliser pour réassembler les données. Ce qui provoque des erreurs de mémoire, la corruption de données ou encore le redémarrage de la machine. source :

2.5 Apache2

2.5.1 Effets des paramètres Apache2

MaxRequestWorkers :

Ce paramètre permet de gérer le nombre maximal de Workers qu'Apache2 peut utiliser simultanément sur des requêtes http.

Lorsque tous les Workers sont occupés, la requête attendra qu'une place se libère et ensuite elle pourra être traitée. Si ce paramètre est faible, le nombre de requêtes pouvant être traitées simultanément sera aussi faible. Cela peut provoquer de la latence chez le client si un nombre important de requêtes sont envoyées simultanément.

À l'inverse si ce paramètre est élevé, le nombre de requêtes pouvant être traitées sera élevé. Si la machine utilisée ne possède pas assez de RAM, celle-ci pourra être surchargée ce qui peut provoquer l'arrêt de certains processus voire de la machine.

C'est pour cela que MaxRequestWorkers doit être paramétrée en fonction des ressources disponibles, pour éviter une surcharge et l'arrêt de certaines fonctions. Et ne pas oublier de prendre en compte le nombre potentiel de requêtes pouvant être envoyées simultanément, pour éviter de créer une latence chez le client

ThreadsPerChild

Un thread est une unité d'exécution permettant à Apache de traiter des Requêtes http. Un processus peut utiliser plusieurs thread simultanément. Le paramètre ThreadsPerChild permet limiter le nombre de Threads pouvant être utilisés par processus.

Comme nous l'avons dit précédemment MaxRequestWorkers limite lui aussi le nombre le nombre de Requêtes pouvant être traité. Pour expliquer simple-

ment les Threads permettent de traiter les requêtes http. Quant à MaxRequestWorkers, il autorise le nombre de requête maximal pouvant être traité simultanément.

ServerLimit

Il existe un derniers paramètre lié aux deux derniers : ServerLimit. Ce paramètre est très important car il définit le nombre de processus Child qu'Apache peut créer. Par exemple, si ServerLimit est défini à 4 processus Child. Si chaque processus possède 25 thread défini par le paramètre ThreadsPerChild. Nous avons donc 4 fois 25 ce qui donne 100 Threads possible au maximum.

Comme pour les autres paramètres, si celui est élevé, la RAM utilisée sera elle aussi élevée. Et inversement, si elle est faible, il y aura une possibilité de latence chez le client.

Pour conclure, pourquoi ces trois paramètres sont étroitement liés entre eux. La calibration de chacun de ces paramètres se fait en fonction des deux autres. Il existe une formule très simple pour voir si la configuration peut marcher :

$$\text{ServerLimit} * \text{ThreadsPerChild} = \text{MaxRequestWorkers}$$

Si MaxRequestWorkers est inférieurs à $\text{ServerLimit} * \text{ThreadsPerChild}$ alors la configuration n'est pas bonne.

Prenons un exemple pour mieux comprendre. La configuration de apache2 est : $\text{ServerLimit} = 4$ $\text{ThreadsPerChild} = 25$ $\text{MaxRequestWorkers} = 50$

Cette composition ne peut pas marcher correctement car MaxRequestWorkers n'est pas égal à 100 ($4 * 25$). Il faut donc définir MaxRequestWorkers à 100 ou Baisser ServerLimit à 2.

3 Conclusion

Texte...

A Annexe A : Données complémentaires

Texte...